COMP3211 Software Engineering
Assignment 1

Wang Ruijie 22103808D

October 7, 2024

1 ACM/IEEE SE Code of Ethics and Professional Practice

Clause 2.02. To reduce costs in the development process, software engineers may be asked to choose unauthorized development tools retrieved and retained unethically, such as cracked operating systems.

Software engineers should have a strong sense of copyright and insist to obtain such software from legal channels or choose legally available alternatives. Meanwhile, employers should provide software engineers with a full set of legally obtained development software to avoid violating copyright.

Clause 7.08. Software engineers may work in areas that they are unfamiliar with . For example, backend engineers may be asked to participate in UI design, but they may neglect to seek advice from senior experts in the field due to ignorance or complacency, which have a possible negative impact on the product.

When software engineers reach the limit of their abilities, they should actively seek advice from professionals, and should also provide advice to others who are new to a field. Employers should create a work environment that encourages communication and mutual assistance across departments, and provide training opportunities for employees to span their skills across multiple fields.

2 Software Processes

My team were required to develop a command-line based task management system in COMP2021 with quite limited time, where we implemented functions of adding, deleting, editing, recovering, and printing user-defined tasks.

An agile method is more appropriate. Although the requirements are fixed, the system is relatively complicated. Time is not sufficient, so it is more efficient to first ensure a working software with minimum requirement and instant testing, and then gradually add more features. We could avoid the failure of the entire project that might occur in plan-driven approaches where no testing is performed until the end of the development.

3 Manifesto for Agile Software Development

Individuals and Interactions over Processes and Tools. If a software project heavily relies on individual expertise without standardized processes and tools, personnel changes might reduce development efficiency and even compromise code integrity.

Working Software over Comprehensive Documentation. If a working software grows to be complex without sufficient documentation, many engineering details may be forgotten, leading to difficulty in maintaining.

Customer Collaboration over Contract Negotiation Excessive customer collaboration without clear contract might result in scope creep and repetitive corrections, affecting the final delivery time and costs.

Responding to Change over Following a Plan Frequent shift in development without a clear plan may increased too muck workloads and cause confusion among team members, leading to a failure in project delivery.

4 eXtreme Programming

Problems on Pair Programming on University Students Pair programming is not effective for university students due to inappropriate partner or lack of ego-less attitude. This is mentioned in the subsection *Pair Programming* of *eXtreme Programming and Education*.

Pair programming is a desirable means to improve code quality in project, but the prerequisite is that both of the partners are expected to have similar programming skills and willingness to take the initiative to avoid that one is always passive. Meanwhile, to be in line with students' score-first framework, the positive effects of ego-less and interacting with partners should be emphasized.

I also believe that partners should not stick to a fixed role. Alternating the roles of programmer and reviewer from time to time or allowing partners to program separately for a period on the same goal and then doing pair programming when merging the code, can avoid premature discussions and give both a stronger motivation of participation.

Problems on On-site Teamwork University students can seldom work together on-site because of the lack of a fixed and comfortable shared working space or inflexible schedules. This is mentioned in the subsection *Distribution and Availability* of *eXtreme Programming and Education*.

Though the problem is inevitable, it is necessary to ensure an on-site working session lasting over four hours every four to five days in a project work. Students may work off-site for the most of the time, but the project can still benefit from the intensive short-term on-site collaboration applying agile practices. A large proportion of distributed independent working and a small proportion of on-site working with agile practices can combine individual's thinking and collective discussion, which is more suitable for the situation in university. Virtual collaboration tools including collaborating editor can also be adapted, but it is unable to replace physically on-site working.

5 Requirements Specification

Functional Requirements The system shall allow users to schedule a meeting by selecting a specific time slot of a date, inputting the meeting title and description, and inviting participants by a automatically generated link or email.

Non-functional Requirements Users should be able to create or attend a meeting after three minutes of guidance. After the guidance, the average time to correct a mistake in using should be less than 3 minute by the users. (A mistake in using is defined as a user's operation that does not meet his or her original intention, e.g., typographical mistakes, wrong time selection, etc.)